

DSA Assignment #1: 概论 & Floyd-Warshall 算法

Updated: 2026-09-09 17:49 (GMT+8)

完成学生: 张然博 信息科学技术学院

说明:

截止日期: 前三周作业统一于 9月29日 提交至 Canvas 平台。

内容要求: 每个题目包含: **解题思路** (可选)、**源代码**、**Accepted 截图**、**预估耗时** (可选)。

1. 题目

E27653: Fraction类

<http://cs101.openjudge.cn/pctbook/E02733/>

OOP, <http://cs101.openjudge.cn/pctbook/E27653/>

主要是练习面向对象编程写法, 这样力扣题目, 笔试都没有问题了。机考时候, 不是必须OOP, 能AC就可以。

代码

```
class Fraction:
    def __init__(self, a, b):
        self.a = a
        self.b = b
    def __add__(self, other):
        return Fraction.reduce(Fraction(self.a * other.b + self.b * other.a,
self.b * other.b))
    def __str__(self):
        return f"{self.a}/{self.b}"
    def reduce(self):
        def gcd(x, y):
            while y:
                x, y = y, x % y
            return x
        g = gcd(self.a, self.b)
        return Fraction(self.a // g, self.b // g)

a, b, c, d = map(int, input().split())
print(Fraction(a, b) + Fraction(c, d))
```

代码运行截图 (至少包含有"Accepted")



E190.颠倒二进制位

bit manipulation, <https://leetcode.cn/problems/reverse-bits/>

思路:

逐个枚举 n 的每一位, 每次把 n 的最低位加进 res 里

代码

```
class Solution:
    def reverseBits(self, n: int) -> int:
        res = 0
        for i in range(32):
            res |= (n & 1) << (31 - i)
            n >>= 1
        return res
```

代码运行截图 (至少包含有"Accepted")



E1356.根据数字二进制下 1 的数目排序

bit manipulation, <https://leetcode.cn/problems/sort-integers-by-the-number-of-1-bits/>

思路:

我们可以使用 $x \&= x - 1$ 来去除 x 最右侧的 1，从而统计出 x 的二进制表示中 1 的个数。然后我们可以使用 Python 的 `sorted` 函数对数组进行排序，排序的关键字是一个元组，元组的第一个元素是 1 的个数，第二个元素是数字本身，这样就可以实现按照 1 的个数排序，如果 1 的个数相同，则按照数字本身排序。

代码

```
class Solution:
    def sortByBits(self, arr: List[int]) -> List[int]:
        def cnt1(x) -> int:
            cnt = 0
            while x:
                x &= x - 1
                cnt += 1
            return cnt
        return sorted(arr, key=lambda x: (cnt1(x), x))
```

代码运行截图 (至少包含有"Accepted")



M27300: 模型整理

sortings, AI, <http://cs101.openjudge.cn/pctbook/M27300/>

思路:

首先读入所有模型信息，然后根据模型名称进行分组。对于每个模型名称，按照其大小进行排序，并输出结果。

代码

```
n = int(input())
d = {}
for _ in range(n):
    model = input()
    l = len(model)
    pos = model.find('-')
    modelName = model[:pos]
    modelSize = float(model[pos + 1:l-1])
    if model[l-1] == 'B':
        modelSize *= 1000
    if modelName not in d:
        d[modelName] = []
    d[modelName].append((model[pos + 1:], modelSize))
for modelName in sorted(d.keys()):
    print(modelName, end=': ')
    for idx, model in enumerate(sorted(d[modelName], key=lambda x: x[1])):
        if idx > 0:
            print(', ', end='')
            print(model[0], end='')
    print()
```

代码运行截图 (至少包含有"Accepted")
#53130750提交状态

[查看](#)[提交](#)

状态: Accepted

源代码

```
n = int(input())
d = {}
for _ in range(n):
    model = input()
    l = len(model)
    pos = model.find('-')
    modelName = model[:pos]
    modelSize = float(model[pos + 1:l-1])
    if model[l-1] == 'B':
        modelSize *= 1000
    if modelName not in d:
        d[modelName] = []
    d[modelName].append((model[pos + 1:], modelSize))
for modelName in sorted(d.keys()):
    print(modelName, end=': ')
    for idx, model in enumerate(sorted(d[modelName], key=lambda x: x[1])):
        if idx > 0:
            print(', ', end='')
            print(model[0], end='')
    print()
```

基本信息

#: 53130750

题目: M27300

提交人: 张然博 250

内存: 3680kB

时间: 24ms

语言: Python3

提交时间: 2026-09-0

M1536.排布二进制网格的最少交换次数

greedy, matrix, <https://leetcode.cn/problems/minimum-swaps-to-arrange-a-binary-grid/>

思路：

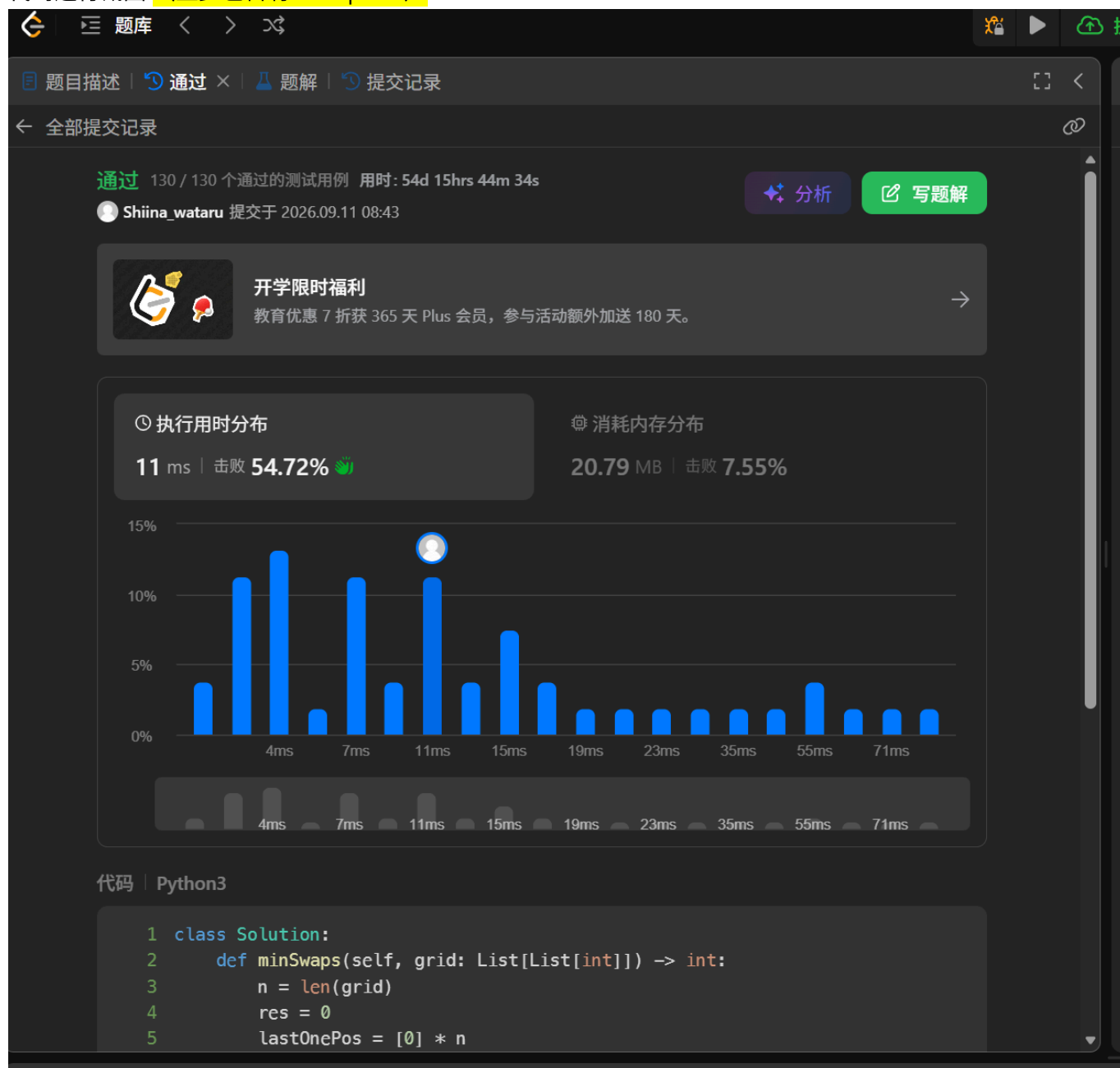
首先，我们需要找到每一行中最右边的1的位置。然后，我们从上到下遍历每一行，对于每一行，我们需要找到最近的一行，其最右边的1的位置小于等于当前行的索引加1。如果找到了这样的行，我们就将其与当前行交换，并记录交换次数。如果没有找到这样的行，说明无法满足条件，返回-1。最后返回总的交换次数。

这个算法的正确性是因为我们总是选择最近的一行进行交换，这样可以保证我们在最少的交换次数内满足条件，并且从需求上来讲我们每次选择的都是需求最迫切的行进行交换。

代码

```
class Solution:
    def minSwaps(self, grid: List[List[int]]) -> int:
        n = len(grid)
        res = 0
        lastOnePos = [0] * n
        def up(i, j) -> None:
            while i < j:
                lastOnePos[j-1], lastOnePos[j] = lastOnePos[j], lastOnePos[j-1]
                j -= 1
        for i in range(n):
            for j in range(n):
                if grid[i][n-1-j] == 1:
                    lastOnePos[i] = n-j
                    break
        for i in range(n):
            for j in range(i, n):
                if lastOnePos[j] <= i+1:
                    res += j - i
                    up(i, j)
                    break
            else:
                return -1
        return res
```

代码运行截图 (至少包含有"Accepted")



05443: 兔子与樱花

<http://cs101.openjudge.cn/practice/05443/>可以使用 **Dijkstra** 算法 (或 Floyd-Warshall 算法) 来求两点之间的最短路

代码

```
from queue import PriorityQueue

nodeList = []
graph = {}
distance = {}

nodeNum = int(input())
for _ in range(nodeNum):
```

```
nodeList.append(input())
roadNum = int(input())
for _ in range(roadNum):
    node1, node2, dis = input().split()
    dis = int(dis)
    if node1 not in graph:
        graph[node1] = []
    if node2 not in graph:
        graph[node2] = []
    graph[node1].append(node2)
    graph[node2].append(node1)
    distance[(node1, node2)] = dis
    distance[(node2, node1)] = dis

def dijkstra(start, end) -> list:
    dist = {node: float('inf') for node in nodeList}
    parent = {node: None for node in nodeList}
    dist[start] = 0
    if start == end:
        return [start,]
    pq = PriorityQueue()
    pq.put((0, start))
    while not pq.empty():
        curDist, curNode = pq.get()
        if curNode == end:
            break
        for neighbor in graph[curNode]:
            newDist = curDist + distance[(curNode, neighbor)]
            if newDist < dist[neighbor]:
                dist[neighbor] = newDist
                parent[neighbor] = curNode
                pq.put((newDist, neighbor))

    path = []
    node = end
    while node is not None:
        path.append(node)
        node = parent[node]
    return path[::-1]

needNum = int(input())
for _ in range(needNum):
    start, end = input().split()
    path = dijkstra(start, end)
    l = len(path)
    for idx, node in enumerate(path):
        if idx == l - 1:
            print(node)
        else:
            nextNode = path[idx + 1]
            print('%s->(%d)->' % (node, distance[(node, nextNode)]), end='')
```

代码运行截图 (至少包含有"Accepted") 

2. 学习总结和收获

如果作业题目简单，有否额外练习题目，比如：OJ“计概2025fall每日选做”、CF、LeetCode、洛谷等网站题目。

额外进行了 LeetCode 上的练习题目：

1. 240.搜索二维矩阵 II
2. 200.岛屿数量
3. 994.腐烂的橘子
4. 207.课程表

额外进行了 每日一题 上的练习题目